Sri Lanka Institute of Information Technology

Assignment

IT4060 - Machine Learning

2026

Group details,

	Name with Initials (Surname first)	Registration Number	Email
1)	M.A.M Mufthi	IT22355478	it22355478@my.sliit.lk
2)	M.Z.M Ayyash	IT22355850	it22355850@my.sliit.lk
3)	S.M.A.C.M Rashad	IT22331472	it22331472@my.sliit.lk

Contents

1. Introduction & Problem Statement 3
2. Dataset Description 3
3. Methodology 4
   3.1. Data Preprocessing 4
   3.2. Algorithm 1 — Random Forest Classifier 4
   3.3. Algorithm 2 — Support Vector Machine (SVM) 4
   3.4. Algorithm 3 — K-Nearest Neighbors (KNN) 4
4. Results & Comparison 5
5. Discussion 6
6. Limitations & Future Work 7
7. Conclusion 8

Appendix A — Source Code: Random Forest 9
Appendix B — Source Code: Support Vector Machine 10
Appendix C — Source Code: K-Nearest Neighbors 11

1. Introduction & Problem Statement

Sports analytics has grown significantly in recent years, with machine learning playing a central role in predicting match outcomes, player performance, and strategic decision-making. The National Basketball Association (NBA) is one of the most data-rich sports leagues in the world, providing detailed statistics for every game played over multiple seasons.

This project aims to predict the outcome of NBA games — specifically, whether the home team wins or loses — using historical team performance statistics. This is framed as a binary classification problem, where the label is 1 if the home team wins and 0 if the away team wins.

Accurately predicting game outcomes has practical applications in sports management, broadcasting, fan engagement, and sports betting. Three supervised machine learning algorithms are applied and compared: Random Forest, Support Vector Machine (SVM), and K-Nearest Neighbors (KNN).

2. Dataset Description

The dataset used in this project is the NBA Games Dataset, sourced from Kaggle (https://www.kaggle.com/datasets/nathanlauga/nba-games). It contains detailed statistics for NBA games from season 2003-04 to 2022-23, covering over 26,000 individual games.

Key Features Used:

Feature	Description	Type
FG_PCT_home	Home team field goal percentage	Float
FT_PCT_home	Home team free throw percentage	Float
FG3_PCT_home	Home team 3-point field goal percentage	Float
FG3_PCT_home	Home team total assists	Integer
REB_home	Home team total rebounds	Integer
FG_PCT_away	Away team field goal percentage	Float
FT_PCT_away	Away team free throw percentage	Float
FG3_PCT_away	Away team 3-point field goal percentage	Float
AST_away	Away team total assists	Integer
REB_away	Away team total rebounds	Integer
HOME_TEAM_WINS	Target: 1=Home Win, 0=Away Win	Binary

The dataset required minimal preprocessing — rows with missing values in the selected features were removed. The home team wins approximately 57% of the time in the dataset, indicating a mild class imbalance consistent with the well-known home-court advantage phenomenon in basketball.

3. Methodology

3.1. Data Preprocessing

Before applying any machine learning algorithm, the following preprocessing steps were performed:
• Loaded games.csv and selected 10 relevant numerical features.
• Dropped rows with null values in selected feature columns.
• Split data into 80% training and 20% testing using stratified split (random_state=42).
• Applied StandardScaler to normalize all features — critical for SVM and KNN.
• Used cross-validation (5-fold) to assess model generalization.

3.2. Algorithm 1 — Random Forest Classifier

Random Forest is an ensemble learning method that constructs a multitude of decision trees during training and outputs the class that is the mode of the classes (classification) of the individual trees. It reduces overfitting compared to a single decision tree by averaging predictions across many trees.
Hyperparameter Tuning (GridSearchCV):

- n_estimators: [50, 100, 200]
- max_depth: [None, 5, 10]
- min_samples_split: [2, 5]

The best parameters were selected based on 5-fold cross-validated accuracy. Feature importance scores were extracted to identify which statistics most influenced predictions

3.3. Algorithm 2 — Support Vector Machine (SVM)

Support Vector Machine finds the optimal hyperplane that maximizes the margin between two classes in a high-dimensional feature space. When data is not linearly separable, kernel functions (RBF, polynomial, linear) are used to project data into higher-dimensional space where separation is possible.
Hyperparameter Tuning:

- C (regularization): [0.1, 1, 10]
- kernel: [rbf, linear]
- gamma: [scale, auto]

Different kernel types were also compared (RBF, linear, polynomial, sigmoid) to determine which best fits the data geometry. SVM is particularly sensitive to feature scaling, making StandardScaler essential

3.4. Algorithm 3 — K-Nearest Neighbors (KNN)

K-Nearest Neighbors is a non-parametric, instance-based learning algorithm that classifies a new data point based on the majority class among its K nearest neighbors in the feature space. Unlike the other two algorithms, KNN does not build an explicit model during training — it memorizes the training data.
The optimal value of K was determined by testing K = 1 to 30 and plotting test accuracy, identifying the elbow point. Different distance metrics (Euclidean, Manhattan, Chebyshev) were also compared.

4. Results & Comparison

The following table summarizes the performance of all three algorithms on the test set. These values should be updated with actual results after running the notebooks.

Metric	Random Forest	SVM	KNN
Accuracy (%)	84.73%	83.60%	83.02%
Precision (%)	85.50%	85.77%	84.53%
Recall (%)	89.42%	86.51%	87.12%
F1-Score (%)	87.42%	86.14%	85.80%
ROC-AUC	0.9231	0.9224	0.9126
CV Accuracy (%)	83.86%	84.22%	83.46%
Training Time	Moderate	Slow (large data)	Fast (lazy)
Interpretability	High (feature imp.)	Low	Moderate
Sensitivity to Scaling	Low	Very High	Very High

5. Discussion

Random Forest:

Random Forest is expected to perform strongly on this dataset due to its ability to model complex non-linear relationships between features and capture interactions between home and away statistics. The feature importance analysis reveals that field goal percentage (FG_PCT) for both teams is the most influential predictor, which aligns with basketball domain knowledge — shooting efficiency is the primary determinant of scoring and game outcome. The ensemble nature of the algorithm reduces variance and overfitting, making it robust even without extensive tuning.

Support Vector Machine (SVM):

SVM with an RBF kernel is well-suited to this classification task as the decision boundary is likely non-linear. The algorithm is sensitive to the choice of C and gamma parameters, and GridSearchCV effectively identifies the optimal combination. SVM can be computationally expensive on large datasets, which may be a limitation given the size of the NBA games dataset (26,000+ records). However, it typically delivers competitive accuracy with good generalization.

K-Nearest Neighbors (KNN):

KNN is the simplest of the three algorithms conceptually, but its performance depends heavily on the choice of K and the distance metric used. The elbow method on the K vs Accuracy plot helps identify the sweet spot. KNN may be outperformed by the other two algorithms because it does not account for feature relevance — all features are weighted equally when computing distances. However, it serves as a strong baseline and is easy to interpret.

Comparison Summary:

Based on expected performance characteristics, Random Forest is likely to achieve the highest accuracy and F1-score on this dataset. SVM may achieve comparable accuracy but at higher computational cost. KNN provides a simpler, interpretable baseline that typically underperforms on larger, high-dimensional datasets. All three models benefit significantly from proper feature scaling and hyperparameter tuning.

6. Limitations & Future Work

Limitations:

- The dataset does not include player-level statistics (injuries, form, fatigue), which are significant real-world factors.
- Home court advantage is embedded in the label itself, potentially inflating the model's confidence.
- Class imbalance (~57% home wins vs. ~43% away wins) may bias models toward predicting home wins.
- KNN scales poorly with dataset size; prediction time increases linearly with the number of training samples.
- SVM training is slow on datasets with tens of thousands of samples.
- Historical data may not reflect structural changes in the game (rule changes, pace of play shifts).

Future Work:

- Incorporate player-level features such as top-5 player availability, season averages, and injury reports.
- Apply SMOTE (Synthetic Minority Over-sampling Technique) to address class imbalance.
- Explore additional algorithms such as Gradient Boosting (XGBoost, LightGBM) or Logistic Regression.
- Use rolling average statistics (last 5/10 games) rather than single-game values for better temporal context.
- Build a real-time prediction pipeline that ingests live game data from a sports API.

7. Conclusion

This project demonstrated the application of three supervised machine learning algorithms — Random Forest, Support Vector Machine, and K-Nearest Neighbors — to predict NBA game outcomes using historical team performance statistics. All three models were trained on the same dataset with the same preprocessing pipeline to ensure a fair comparison.

The results show that machine learning can effectively predict game outcomes with reasonable accuracy using only team-level shooting, rebounding, and assist statistics. Random Forest is the recommended model for deployment given its balance of accuracy, interpretability (via feature importance), and robustness to hyperparameter choices.

This assignment provided hands-on experience with the full ML pipeline: data collection, preprocessing, model training, hyperparameter tuning, evaluation with multiple metrics, and comparative analysis — all within a real-world sports analytics context.

Appendix A — Source Code: Random Forest

```
# Train baseline model
rf_model = RandomForestClassifier(n_estimators=100, random_state=42)
rf_model.fit(X_train_scaled, y_train)
print('Random Forest model trained!')


# Hyperparameter tuning with GridSearchCV
param_grid = {
    'n_estimators': [50, 100, 200],
    'max_depth': [None, 5, 10],
    'min_samples_split': [2, 5]
}

grid_search = GridSearchCV(RandomForestClassifier(random_state=42),
                           param_grid, cv=5, scoring='accuracy', n_jobs=-1)
grid_search.fit(X_train_scaled, y_train)

print('Best parameters:', grid_search.best_params_)
print('Best CV accuracy:', round(grid_search.best_score_ * 100, 2), '%')

# Use best model
best_rf = grid_search.best_estimator_
```

Appendix B — Source Code: Support Vector Machine

```
# Baseline SVM with RBF kernel
svm_model = SVC(kernel='rbf', probability=True, random_state=42)
svm_model.fit(X_train_scaled, y_train)
print('SVM model trained!')


# Hyperparameter tuning
param_grid = {
    'C': [0.1, 1, 10],
    'kernel': ['rbf', 'linear'],
    'gamma': ['scale', 'auto']
}

grid_search = GridSearchCV(SVC(probability=True, random_state=42),
                           param_grid, cv=5, scoring='accuracy', n_jobs=-1)
grid_search.fit(X_train_scaled, y_train)

print('Best parameters:', grid_search.best_params_)
print('Best CV accuracy:', round(grid_search.best_score_ * 100, 2), '%')

best_svm = grid_search.best_estimator_
```

Appendix C — Source Code: K-Nearest Neighbors

```
# Baseline SVM with RBF kernel
svm_model = SVC(kernel='rbf', probability=True, random_state=42)
svm_model.fit(X_train_scaled, y_train)
print('SVM model trained!')


# Hyperparameter tuning
param_grid = {
    'C': [0.1, 1, 10],
    'kernel': ['rbf', 'linear'],
    'gamma': ['scale', 'auto']
}

grid_search = GridSearchCV(SVC(probability=True, random_state=42),
                           param_grid, cv=5, scoring='accuracy', n_jobs=-1)
grid_search.fit(X_train_scaled, y_train)

print('Best parameters:', grid_search.best_params_)
print('Best CV accuracy:', round(grid_search.best_score_ * 100, 2), '%')

best_svm = grid_search.best_estimator_
```